In dictionary default value is None.

Dictionary Datatype

Dictionary is a combination of key & value pair.

The boundary condition of dictionary is { }

Internally we represent as object dict()

without object we create empty dict as.

```
a = { }
    { }
```

with object `a = dict()`
`{ }`

Mutable data type

Syntax:

```
vn = { key1 : value, key2 : value , ... }
```

In place of key we have to use only single value data & immutable data

Inplace of value we used any data type in python.

key :- Single value data, immutable data (string, tuple)

value :- Any data type in python.

Mutable dic, set, list
immutable string, tuple

Another name of tuple data type is composite key

Page No.
Date

Note:-

In key part if we pass mutable data type. (dic, set, list) it will show unhashable error. (mutable d.T. error) key error.

- We can't do indexing & slicing because each element is seperated by collen (but indirectly we do)
- In key part if you want to pass multiple value at a time then use parenthesis

eg:

```
W = {(1,2,3) : 100}
     ↑
  composite key.
```

Main Difference

list	Tuple.
We can not use in key part	We can used in key part

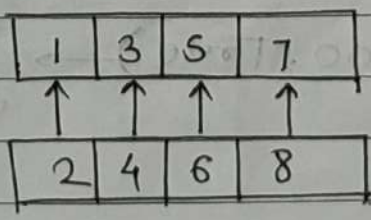
Eg:

```
a = {1:100, "python":"HTML"}
len(a)
2. (length always count in pair)
```

How to access value in dictionary data type?

```
x = {1:2, 3:4, 5:6, 7:8}
```

internally



→ visible layer while doing operation.

Because all values depends on key

Methods

How to access particular value outside dictionary.

1) Without using inbuilt function.

① key & value | Syntax :- vn[key]

eg: a = { 100:100, 200:250, 400:3 }

a[200]

250

If key is not present in dict then we get keyError.

2) with using inbuilt function.

② i) get()

Syntax :-

vn.get(key, default value)

- If key is not present it give blank space in output no error.

a.get(100)

100

a.get(1000)

a.get(1000, True) → give output which we provide in default value.

'True'

③ 2) setdefault()

Syntax :

vn.setdefault(key, default value)

- If key is not present then it will not give any error it will add that key into last of that dict and None as value.

eg:-

a.getdefault(100)

100

a.Setdefault(1000) → Not present no error.

a

{100:100, 200:250, 400:3, 1000:None}

a.set default(1000, True)

↳ if default value is given then present.

a

{100:200, 200:250, 400:3, 1000:True}

④ To access all keys:

vn.keys()

a.keys()

dict-keys([100, 200, 400, 1000])

⑤ To access all values:-

vn.values() / a.values()

dict_values([100, 250, 3, None])

⑥ To access all items:-

vn.items()

a.items()

dict_items([(100:200), (200:250), (400:3), (1000:None)])

Page No.
Date

For Adding.

⑦ without inbuilt function.

Syntax:

```
vn[key] = value
```

- It add one pair at a time.

```
a={ }
a[100] = 50
a
{100:50}
```

⑧ with inbuilt function. update().

Syntax:

```
vn.update({key:value})
```

It add multiple pair at a time.

```
a = { }
a.update({90:5, 60:3, 20:4})
a
{90:5, 60:3, 20:4}
a.update({5:2, 5:3})
                └ same key so it take recently updated value
a
{90:5, 60:3, 20:4, 5:3}
```

Deleting purpose:

(9) Pop()

Syntax :- `vn.pop(key, 'default value')`

mandatory field.

```
a={100:200, 4:500}
a.pop()
 Type error.
a.pop(600)
 key error
a.pop(600,'True')
 True
a.pop(100)
200
a
{4:500}
```

It is used to delete specific position key & value.

(10) popitem()

Syntax :- `vn.popitems()`

It is used to remove last element from dict.

```
a={100:200, 4:500}
a.popitem()
(4:500)
```

output in tuple form.

(11) clear()

Syntax :- `vn.clear()`

It we want to clear all dict element

```
a.clear()
{}
```

If we want to remove dict from its memory location that time we use del vn.

del a

copy()

Copy is process of copying one variable into another variable

Syntax:- new vn = original vn.

the id of both new & original dict are same.

```
a = {10:50}
b = a
a
{10:50}
b
{10:50}
id(a)
2315310202 88
id(b)
2315310202 88
```

fromkeys()

It is the process of converting one datatype into ~~one~~ dictionary D.T.

Syntax:-

dict.fromkeys(iterable, fillvalue)

It does not modify the original dictionary.

```
a = { 40 : 50 , 70 : 20 }
dict.fromkeys(a)
{' 40 : None , 70 : None }
dict.formkeys(a, 'me')
{40 : 'me' , 70 : 'me'}
a
{ 40 : 50 , 70 : 20 }
```

Dictionary merging :

```
a = {1:2}
b = {2:4}
c = {*a, *b}
c
{1, 2}
```

if we are using only single * we get only keys.

```
c = {**a, **b}
c
{1:2 , 2:4}
```

if we are using two * we merge both key & value.

```
a|b
{1:2 ; 2:4 }
```

if we use pipeline (|) symbol also we can merge dictionary.